ISIT312 Big Data Management

Spark Data Model

Dr Guoxin Su and Dr Janusz R. Getta

School of Computing and Information Technology -
University of Wollongong

Introduction to Spark

Outline

Resilient Distributed Data Sets (RDDs)

Resilient Distributed Datasets (RDDs) is the lowest level (and the oldest) data abstraction available to the users

An RDD represents an immutable, partitioned collection of elements that can be operated on in parallel

Every row in an RDD is a Java object

RDD does not need to have any schema defined in advance

It makes RDD very flexible for various applications but in the same moment makes the manipulations on data more complicated

Users must implement their own functions to perform simple tasks like for example aggregation functions: average, count, maximum, etc

RDDs provide more control on how data is distributed over a cluster and how it is operated

Resilient Distributed Data Sets (RDDs)

RDD is characterized by the following properties

- A list of partitions
- A function for computing each split
- A list of dependencies on other RDDs
- Optionally, a Partitioner for `key-value` RDDs
- Optionally, a list of preferred locations to compute each split

Specification of custom partitions may provide significant performance improvements when using `key-value` RDDs

The properties of RDDs determine how Spark schedules and processes the applications

Different RDDs may use different properties depending on the required outcomes

Resilient Distributed Data Sets (RDDs)

Every Spark script includes an object `SparkContext` that connects to the cluster and allows processing to be parallelized

Typical creation of `SparkContext` object

SparkContext object

```
sc = SparkContext(appName="RDD Examples")
```

When connecting from `pyspark` shell `SparkContext` object is created automatically

RDD can be created from a list

Creating RDD from a list of numbers

```
numbers = sc.parallelize([1, 2, 3, 4, 5])
```

Creating RDD from a list of strings

```
names = sc.parallelize(["James","Harry", "Robin"])
```

Listing RDD

Listing RDD

```
names.collect()
print(names.take(3))
```

Resilient Distributed Data Sets (RDDs)

Assume that a file `NOTICE.txt` is localed in `/user/bigdata` folder in HDFS and it is a home folder of `bigdata` user

Creating RDD from a file in HDFS

Creating RDD from a file in HDFS

```
notice_lines = sc.textFile("NOTICE.txt")
```

Assume that a file `README.txt` is localed in `/user` folder in HDFS on the local host

Creating RDD from a file in HDFS

Creating RDD from a file in HDFS

```
read_lines = sc.textFile("hdfs://localhost:9000/user/README.txt")
```

Saving RDD in HDFS

Saving RDD in HDFS

```
read_lines.saveAsTextFile("hdfs://localhost:9000/user/readme")
```

Contents of HDFS

```
hdfs dfs -ls /user/readme
Found 2 items
-rw-r--r--   1 bigdata supergroup          0 2026-04-15 19:00 /user/readme/_SUCCESS
-rw-r--r--   1 bigdata supergroup        175 2026-04-15 19:00 /user/readme/part-00000
```

Resilient Distributed Data Sets (RDDs)

Assume that a file `words.txt` is localed in `/user/bigdata` folder in HDFS and it is a home folder of `bigdata` user

A file `words.txt` contains the words separated with blanks

The following program computes the count of each word in a file `words.txt`

Creating lines

```
lines_rdd = sc.textFile("words.txt")
```

Counting words

```
counters = (lines_rdd.flatMap(lambda line: line.split(" "))
                     .map(lambda word: (word, 1))
                     .reduceByKey(lambda a, b: a + b) )
```

Results

```
counters.collect()
```

Introduction to Spark

Outline

Resilient Distributed Data Sets (RDDs)

DataFrames

SQL Tables/Views

Datasets

DataFrames

A DataFrame is a table of data with rows and columns

A list of columns in DataFrame together with the types of columns is called as a schema

A DataFrame can span over many systems in a cluster

The concepts similar to DataFrame are used in Python and R

A DataFrame is the simplest data model available in Spark

A DataFrame consist of zero or more partitions

- To parallelize data processing all data are divided into chunks called as partitions
- A partition is a collection of rows located on a single system in a cluster
- When operating DataFrame Spark simultaneously processes all relevant partitions
- Shuffle operation is performed to share data among the systems in a cluster

DataFrames

Working with DataFrames starts from creating Spark Session

Spark Session

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Demo").getOrCreate()
```

A DataFrame can be created in the following way

Creating a DataFrame

```
people = spark.createDataFrame([("James", 25),("Harry",17),("Robin",40)],["name","age"])
```

A DataFrame can be listed in the following way

Listing a DataFrame

```
people.show()
```

Vusualisation of DataFrame

```
+-----+---+
| name|age|
+-----+---+
|James| 25|
|Harry| 17|
|Robin| 40|
+-----+---+
```

DataFrames

A schema of a DataFrame can be listed in the following way

Listing a schema

```
people.printSchema()
```

Results

```
root
 |-- name: string (nullable = true)
 |-- age: long (nullable = true)
```

It is possible to add a column an already existing DataFrame

Adding a column

```
new_people = people.withColumn("life_stage", when(col("age") < 13, "child")
...                                          .when(col("age").between(13, 19), "teenager")
...                                          .otherwise("adult") )
```

Listing a DataFrame

```
new_people.show()
```

Results

```
+-----+---+----------+
| name|age|life_stage|
+-----+---+----------+
|James| 25|     adult|
|Harry| 17|  teenager|
|Robin| 40|     adult|
+-----+---+----------+
```

In HTML view press 'p' to see the lecture notes

DataFrames

Creating a DataFrame in a different way

Creating a DataFrame

```
people = spark.createDataFrame([{"name":"James","age":25},
...                             {"name":"Harry","age":24},
...                             {"name":"Robin","age":50}])
```

Listing a DataFrame

```
people.show()
```

Results

```
+---+-----+
|age| name|
+---+-----+
| 25|James|
| 24|Harry|
| 50|Robin|
+---+-----+
```

Listing a schema

```
people.printSchema()
```

Results

```
root
 |-- age: long (nullable = true)
 |-- name: string (nullable = true)
```

DataFrames

Assume that the files `people.json` and `people.txt` are uploaded to HDFS

```
hdfs dfs -cat /user/bigdata/people.json
{"name":"James","age":25}
{"name":"Harry","age":24}
{"name":"Robin","age":50}
```
people.json

```
hdfs dfs -cat /user/bigdata/people.txt
james,23
john,34
lisa,23
```
people.txt

Creating a DataFrame from a file `people.json`

```
people = spark.read.json("people.json")
```
people.json

```
people.show()
+---+-----+
|age| name|
+---+-----+
| 25|James|
| 24|Harry|
| 50|Robin|
+---+-----+
```
Results

DataFrames

Creating a DataFrame from a file `people.txt`

people.txt

```
people = spark.read.text("people.txt")
```

Results

```
people.show()
+--------+
|   value|
+--------+
|james,23|
| john,34|
| lisa,23|
+--------+
```

One more way of creating a DataFrame

Import Row

```
from pyspark.sql import Row
```

Create DataFrame

```
df = spark.createDataFrame([Row(name="James", age=25)])
```

Results

```
df.show()
+-----+---+
| name|age|
+-----+---+
|James| 25|
+-----+---+
```

DataFrames

Saving a DataFrame in HDFS

Creating a folder in HDFS

```
hdfs dfs -mkdir /user/bigdata/people
```

Saving DataFrame in HDFS in csv format

```
people.write.save("/user/bigdata/people", format="csv", mode="append")
```

Listing the contents of a folder

```
hdfs dfs -ls /user/bigdata/people/*
-rw-r--r--   1 bigdata supergroup          0 2026-04-17 10:21 /user/bigdata/people/_SUCCESS
-rw-r--r--   1 bigdata supergroup         27 2026-04-17 10:21 /user/bigdata/people/
                                                part-00000-ab78decf-d976-4bcb-b2c9-1e9f7c178fa5-c000.csv
```

Listing the contents of a file

```
hdfs dfs -cat /user/bigdata/people/part*
James,25
Harry,24
Robin,50
```

Introduction to Spark

Outline

SQL Tables/Views

SQL can be used to operate on DataFrames

It is possible to register any DataFrame as a table or view (a temporary table) and apply SQL to it

There is no performance overhead when registering a DataFrame as SQL Table and when processing it

A DataFrame `dataFrame` can be registered as SQL temporary view `people` in the following way

Creating temporary view

```
people.createOrReplaceTempView("people_table")
```

Accessing a temporary view in SQL

```
spark.sql("SELECT * from people_table").show()
+---+-----+
|age| name|
+---+-----+
| 25|James|
| 24|Harry|
| 50|Robin|
+---+-----+
```

SQL Tables/Views

Accessing a temporary view in SQL

Accessing a temporary view in SQL

```
spark.sql("SELECT count(*) from people_table").show()
+--------+
|count(1)|
+--------+
|       3|
+--------+
```

Accessing a temporary view in SQL

```
spark.sql("SELECT * from people_table WHERE age>25").show()
+---+-----+
|age| name|
+---+-----+
| 50|Robin|
+---+-----+
```

Saving results as a DataFrame

```
table_df = spark.sql("SELECT * from people_table WHERE age>25")
table_df.show()
+---+-----+
|age| name|
+---+-----+
| 50|Robin|
+---+-----+
```

Introduction to Spark

Outline

Datasets

A Dataset is a distributed collection of data

A DataFrame, is a Dataset of type `Row`

Datasets consists of the objects included in the rows; one object per row

Datasets are a strictly JVM language feature that only work with Scala and Java

Python does not have the support for the Datasets

A Dataset can be constructed from JVM and then manipulated using functional transformations

In Scala a `case class` object defines a schema of an object

Datasets use a specialized Encoder to serialize the objects for processing or transmitting over the network

Dataset supports all operations of DataFrame

Datasets

A Dataset can be defined, created, and used in the following way

Creating Dataset Schema

```
case class Person(name: String, age: Long)
```

Creating Dataset

```
val caseClassDS = Seq(Person("Andy", 32)).toDS()
```

Listing Dataset

```
caseClassDS.show()
```

Results

```
+----+---+
|name|age|
+----+---+
|Andy| 32|
+----+---+
```

Using a Dataset

```
caseClassDS.select($"name").show()
```

Results

```
+-----+
| name|
+-----+
|James|
+-----+
```

References

A Gentle Introduction to Spark, Databricks, (Available in `READINGS` folder)

RDD Programming Guide

Spark SQL, DataFrames and Datasets Guide

Karau H., Fast data processing with Spark Packt Publishing, 2013 (Available from UOW Library)

Srinivasa, K.G., Guide to High Performance Distributed Computing: Case Studies with Hadoop, Scalding and Spark Springer, 2015 (Available from UOW Library)

Chambers B., Zaharia M.,Spark: The Definitive Guide, O'Reilly 2017